
An Analysis of Model Compression Techniques

Ahsan Usman* Faaiz Umar* Haider Khan*

Abstract

The growing scale of deep neural networks limits deployment on edge devices. Model compression, through pruning, quantization, and knowledge distillation, reduces memory and latency while maintaining accuracy. Unstructured pruning can recover dense-model accuracy after fine-tuning, whereas structured pruning achieves real latency and MAC reductions with minimal loss. Quantization compresses by lowering numerical precision instead of removing weights. Mixed-precision and Hessian-guided quantization recover FP32-level accuracy at a fraction of the model size. For knowledge distillation, distilling VGG-16 to VGG-11 on CIFAR-100 reveals that matching internal features (Hints) yields the best student accuracy (67.73%), while logit matching maximizes fidelity (KL: 0.1178), exposing a trade-off between performance and mimicry. Code and reproducibility material are available at: [code](#).

1. Introduction

Large deep neural networks are typically over-parameterized relative to the complexity of their downstream tasks. While this overparameterization aids optimization and generalization, it also leads to models that are difficult to deploy on resource-constrained devices. A lot of work therefore studies model compression, which encompasses (i) pruning, (ii) quantization and (iii) knowledge distillation.

Pruning aims to remove unimportant parameters or channels (Han et al., 2015) based on certain importance thresholds. We study magnitude-based fine-grained pruning and channel-based structured pruning by utilizing a VGG-11 fine-tuned on CIFAR-10 as a controlled testbed. We follow the prune, evaluate, finetune protocol described by (Han et al., 2015) and analyze task performance latency, MAC counts, weight distributions, and Grad-CAM maps to under-

stand how pruning affects the model. We show unstructured pruning allows for high degrees of sparsity with essentially no drop in accuracy at the cost of limited gains in latency and MACs. Structured pruning, however, shows significant gains by reducing latency and memory footprint at the cost of very minimal performance drops, highlighting its effectiveness for resource-constrained environments.

Quantization reduces model precision to lower bit-width representations, shrinking memory footprint and improving inference efficiency without structural changes. We benchmarked post-training (PTQ) and quantization-aware training (QAT) across multiple bit widths (FP16, BF16, INT8, INT4) on VGG-11 and ResNet models using CIFAR-100. While FP16 and BF16 preserved near-baseline accuracy, uniform INT quantization caused degradation, especially at INT4. QAT recovered much of this loss, and mixed-precision quantization achieved FP32-level accuracy. Quantization is complementary to pruning by reducing numerical precision rather than weights.

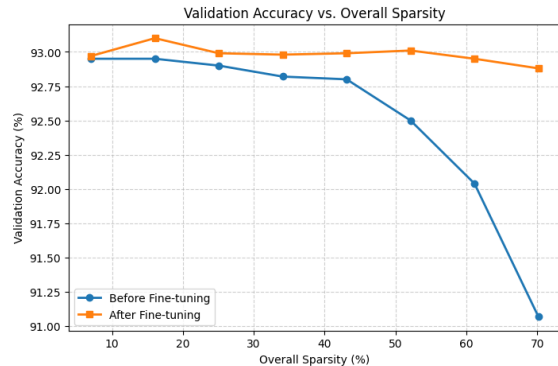


Figure 1. Effect of sparsity on model accuracy, pre- and post-fine-tuning. This is done for unstructured pruning utilizing the ratios from the sensitivity analysis and scaling it with multiplicative factors to achieve a wide range of sparsity levels.

Knowledge Distillation (KD) compresses large teachers into smaller students, but the optimal knowledge to transfer (logits vs. internal features) is debatable. Distilling VGG-16 to VGG-11 on CIFAR-100 with various methods (LM, DKD, Hints, CRD), we find a critical trade-off: matching internal, spatial features (Hints) yields the best student performance (67.73%), while matching final logits (Logit Match-

*Equal contribution . Correspondence to: Ahsan Usman <26100009@lums.edu.pk>, Faaiz Umar <26100088@lums.edu.pk>, Haider Khan <26100053@lums.edu.pk>.

ing) yields the best fidelity but lower accuracy. This shows a disconnect between knowledge for accuracy versus mimicry. We also analyze the incidental transfer of localization and invariance, finding teacher capacity is not the sole predictor of efficacy.

2. Methodology

2.1. Pruning

To evaluate different pruning strategies, we use the VGG-11 (Simonyan & Zisserman, 2015) architecture fine-tuned on the CIFAR-10 (Krizhevsky, 2012) dataset as the baseline for comparison. In our experimental setup, we compare two main pruning strategies: (i) *unstructured pruning* (ii) *structured pruning*. We follow the overall methodological framework described in (Han et al., 2015). Across all experiments, the target overall sparsity level/pruning ratio is set to $\alpha = 0.7$.

Unstructured Pruning: For unstructured pruning, we utilize a magnitude-based pruning strategy (Han et al., 2015). The importance of each weight is measured by its magnitude, and weights below a certain global threshold are pruned (set to zero) to achieve the target sparsity level α . Formally, the pruning process can be expressed as:

$$W_{\text{new}} = W_{\text{orig}} \odot M, \quad M = |W_{\text{orig}}| > v_{\text{thresh}}, \quad (1)$$

$$\alpha = 1 - \frac{\|W_{\text{new}}\|_0}{\|W_{\text{orig}}\|_0}, \quad (2)$$

where W_{orig} denotes the original weight tensor, $M \in \{0, 1\}$ represents the binary pruning mask, and v_{thresh} is the threshold determined such that the desired sparsity α is met. We first perform a sensitivity analysis by pruning one layer at a time (keep all others fixed) at varying sparsity levels and measuring the accuracy drop. This analysis helps identify layers that are more sensitive to pruning and thus require lower sparsity thresholds. Based on this insight, and that layers with more parameter can have higher sparsity, we select layer-wise sparsity levels to maximize accuracy retention and prune the dense model. After pruning, we test the model’s test accuracy and then fine-tune it for 5 epochs SGD with a cosine annealing learning rate scheduler to recover the drop in accuracy. Lastly, to study the trade-off between sparsity and performance, we scale the layer-wise thresholds by a range of multiplicative factors to analyze the effect of sparsity on both pre- and post-fine-tuning accuracies.

Structured Pruning: For structured pruning, we perform channel pruning, which directly affects the dimensionality of the weight tensors by reducing the number of output/input channels. Each convolutional layer’s importance is determined through the Frobenius norm and the top-k channel weights are kept to meet the prune ratio. We conduct a similar sensitivity analysis for structured pruning, focusing on

convolutional and batch normalization layers while excluding the classifier. The inter-layer sparsity ratio is varied for selected layer pairs, keeping others constant, to determine their sensitivity. We then utilize the results from this to select pruning ratios (how many channels to keep) to ensure the model size is 30% of the original. Then, the pruned model is evaluated, fine-tuned and then evaluated again.

Evaluation: The dense, unpruned VGG-11 is first evaluated in terms of its test accuracy, number of multiply-accumulate operations (MACs), average inference latency (computed over 20 runs), and model storage size. Finally, we visualize weight distributions before and after pruning and generate Grad-CAM visualizations from the last layer in the backbone to qualitatively assess the effect of pruning (Selvaraju et al., 2019). This process is repeated for all pruning strategies to allow for a consistent comparison.

2.2. Quantization

We used the CIFAR-100 (Krizhevsky, 2009) dataset comprising of 100 different classes with 50000 training images and 10000 testing images to benchmark different quantization methods on 4 different models: Resnet-56, Resnet-44, Resnet-32, and VGG-11. We got the pretrained versions of these models from chenyaofu (Yao, 2021).

2.2.1. PTQ & QAT

We evaluated Post-Training Quantization (PTQ) on four bit widths, FP16, BF16, INT8, and INT4, and Quantization-Aware Training (QAT) on INT8 and INT4. Since PTQ on FP16 and BF16 showed negligible degradation, QAT was not performed for these formats. Quantization was implemented using TorchAO (Or et al., 2025) with asymmetric per-layer activation quantization (0–255 for INT8, 0–15 for INT4) and symmetric per-channel weight quantization (−128–127 for INT8, −8–7 for INT4). For PTQ calibration, 10 examples per class from the training set were used to compute scale and zero points, while QAT employed the full CIFAR-100 training loader. Models were trained for up to 10 epochs using SGD ($\text{lr} = 1 \times 10^{-3}$, momentum = 0.9, weight decay = 1×10^{-4}), freezing quantization parameters after the first epoch. It is important to note that the INT4 configuration used INT8 quantization with manually constrained ranges, enabling accuracy benchmarking but not speed. Memory requirements were calculated manually for INT4. Floating-point quantization was done via PyTorch’s built-in casting functions (.half, .to(torch.bfloat16)). We report accuracies for all tested bit widths under both PTQ and QAT.

2.2.2. SCALING LAWS

We conducted scaling law analysis to determine how the accuracy of our model changes given different bit widths at

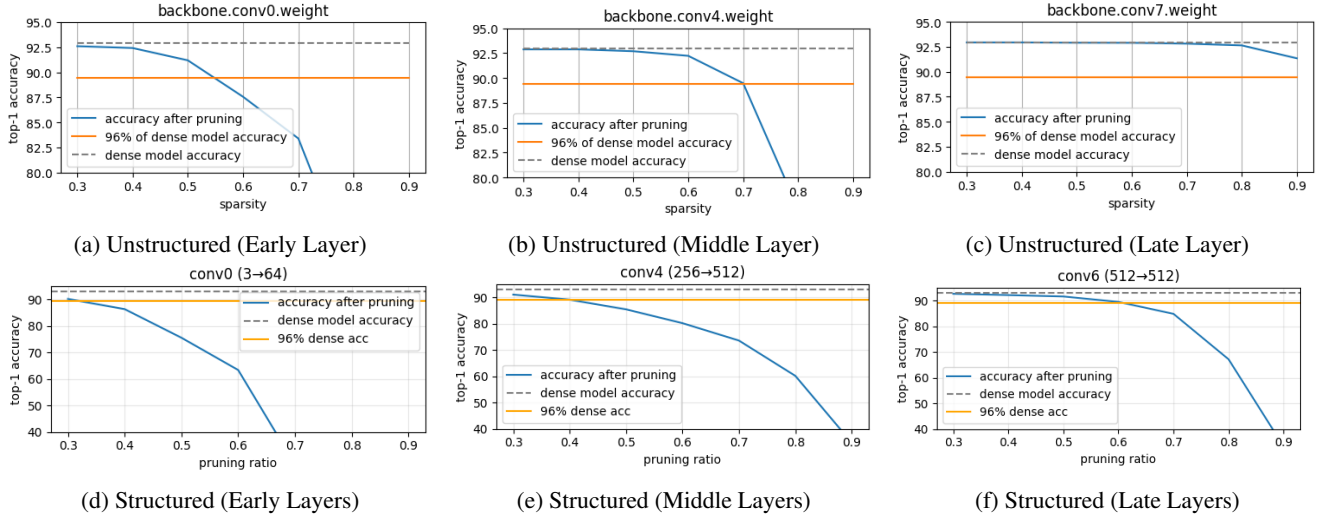


Figure 2. Layer-wise sensitivity analysis for unstructured (top row) and structured (bottom row) pruning. Each plot shows the effect of varying sparsity levels on accuracy for different convolutional layers, highlighting which layers are most sensitive to pruning.

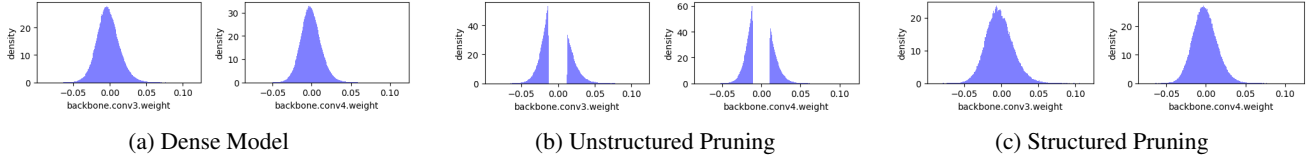


Figure 3. Comparison of weight distributions for the dense, unstructured-pruned, and structured-pruned models.

a particular quantization level. We plotted accuracy against bit widths for all of our models in both PTQ and QAT setup. The paper (Dettmers & Zettlemoyer, 2023) was the closest to our approach. However, they plotted accuracy against the total model bits for each bit width. Given that we benchmarked only 4 models meant that this setup would not give us satisfactory results hence we modified their setup to match our needs.

2.2.3. MIXED PRECISION QUANTIZATION

Some model layers are more sensitive to weight perturbations than others. We leveraged this observation by applying mixed-precision quantization on a VGG-11 model pretrained on CIFAR-100. Our baseline model was uniformly quantized to FP16 and evaluated on the CIFAR-100 test set. We then conducted two further experiments. (1) The first convolutional block and the final linear layer were kept in FP16 precision, while all intermediate convolutional and linear layers were quantized to INT8. (2) We applied the Hessian-Aware Quantization (HAWQ-V2) method (Dong et al., 2020) to determine the relative sensitivity of each layer to quantization noise. We used four calibration batches, each comprising 64 images, to estimate the gradients for the Hessian computation. The number of random Rademacher probes was set to eight to approximate the layer-wise Hessian trace. Based on the average trace

of the Hessian, layers with high sensitivity ($\text{trace} \geq 10^{-1}$) were kept in FP16, moderately sensitive layers ($10^{-3} \leq \text{trace} < 10^{-1}$) were quantized to INT8, and the least sensitive layers ($\text{trace} \leq 10^{-4}$) were quantized to INT4.

Activations were quantized to INT8 (per-tensor affine), while weights followed per-channel symmetric quantization. This setup ensured efficient execution while preserving accuracy in the sensitive layers. We reported accuracy, inference speed, and model size (theoretical and serializable). The theoretical size represents the bit level precision needed by each layer, while the serializable size corresponds to the actual saved checkpoint including quantization metadata.

2.2.4. OUTLIERS IN QUANTIZATION

Outliers can degrade quantized model performance by disproportionately influencing the scaling factors. To visualize potential outliers, we collected activations from all ReLU layers and the final linear layer of the pretrained VGG-11 on CIFAR-100, along with all convolutional and linear weights, and plotted their histograms. To mitigate outlier effects, we implemented a percentile-based clipping strategy within a PTQ framework using the FBGEMM backend. Instead of fixing clipping thresholds manually, we performed a grid search over activation percentiles [99.0, 99.5, 99.7, 99.8, 99.9, 99.95]% and weight

percentiles [99.0, 99.5, 99.9]%, selecting the configuration that minimized mean-squared error between FP32 and INT8 logits on a balanced 1k-image calibration subset.

To further analyze whether outlier sensitivity varies with depth, we divided the quantizable layers into three partitions, early, middle, and late, and applied clipping selectively to each, keeping other layers under standard min-max calibration. Each configuration was evaluated using both MSE and top-1 accuracy, providing insight into which network regions benefit most from localized outlier suppression.

2.3. Knowledge Distillation

We evaluated the transfer of predictive, localization, and invariance knowledge via knowledge distillation (KD) on the CIFAR-100 dataset. The teacher was an ImageNet-pretrained VGG-16, and the student a VGG-11 trained from scratch. Both models’ classifiers were replaced to match the output classes of the dataset. Training data was augmented with random crops, random horizontal flips, and normalization. The primary metric was top-1 accuracy. VGG-16 and 19 teacher were fine-tuned for 25 epochs and the best models were saved and frozen.

We trained the VGG-11 student via several protocols: *Student Independent (SI)*: (Baseline) 50 epochs using standard cross-entropy loss on hard labels. *Label Smoothing (LS)*: (Baseline) 60 epochs, against a smoothed target ($\epsilon = 0.1$) defined as $L_{LS} = \text{mean}(\sum -\text{true_dist} \times \log_probs)$. *Logit Matching (LM)*: 50–60 epochs with $L = (1.0 - \alpha)L_{CE} + \alpha L_{KD}$. We used $\alpha = 0.9$, $T = 4.0$, and T^2 -scaled KL divergence. *Decoupled KD (DKD)*: 60 epochs with $L = L_{CE} + L_{DKD}$. L_{DKD} (at $T = 1.0$) was the sum of a Target Class (TKD) and Non-Target Class (NCKD) binary KL divergence. *Hint-based Distillation (Hints)*: 50 epochs with $L = L_{CE} + 1.0 \times L_{Hint}$. L_{Hint} was MSE between the features via a 1x1 adapter (256 to 512 channels). Gradient clipping (norm 5.0) was applied. *Contrastive (CRD)*: 50 epochs with $L = L_{CE} + 1.0 \times L_{CRD}$. L_{CRD} was a symmetric, in-batch NT-Xent loss ($\tau = 0.2$) on 128-dim projections from the 4096-dim penultimate features.

We employed the following metrics and experimental protocols to quantify knowledge transfer: *Distributional Similarity*: average KL divergence ($KL(T||S)$) on the test set, using softened logits ($T = 4.0$). *Localization Similarity*: cosine similarity between un-resized, flattened GradCAM heatmaps from the teacher’s and student’s Block 3 layers. *Color Invariance*: we created a T_{color} by fine-tuning the VGG-16 for 10 epochs on ColorJitter-augmented data. A student (S_D) was then trained via CRD from T_{color} on non-jittered data. We measured the accuracy drop for all models on a ColorJitter-augmented test set. *Teacher Capacity*: We compared the final accuracy of the LM student trained from the VGG-16 vs. the VGG-19 teacher.

Table 1. Comparison of dense, unstructured-pruned (Unstruc.), and structured-pruned (Struc.) VGG-11 models on CIFAR-10.

Metric	Dense	Unstruc.	Struc.
Acc. (pre FT)	92.95%	91.07%	12.13%
Acc. (post FT)	–	93.04%	91.84%
Latency (ms)	2.53	2.33	0.87
MACs (M)	606.16	606.16	189.47
Parameters	9,228,362	9,228,362	2,644,058
Size (MiB)	35.20	10.50 (29.8%)	10.09 (28.7%)

3. Results

3.1. Pruning

Table 1 summarizes the performance of dense, unstructured-pruned, and structured-pruned VGG-11 models on CIFAR-10. The dense baseline achieves 92.95% accuracy with the highest latency (2.53 ± 0.03 ms). Unstructured pruning at $\alpha = 0.7$ maintains strong performance (91.07%) and, after brief fine-tuning, surpasses the baseline at 93.04% with reduced latency (2.33 ± 0.10 ms). However, this reduction is only theoretical: although the model retains 29.82% of its parameters (10.50 MiB effective size), the physical matrix dimensions remain unchanged. Without specialized sparse hardware, the actual storage remains 35.20 MiB, with parameter count and MACs identical to the dense model. Structured pruning, in contrast, yields substantial computational benefits. After fine-tuning, accuracy recovers to 91.84%, while MACs and latency drop by 68.7% and 65%, respectively, with a 70% smaller memory footprint. Without fine-tuning, accuracy falls drastically to 12.13%, but this is largely recovered through retraining.

Figure 1 shows accuracy versus sparsity for unstructured pruning. Validation accuracy remains stable up to $\alpha \approx 0.4$ but degrades sharply beyond $\alpha = 0.5$ ($> 2\%$ drop). Fine-tuning mitigates this, limiting post-pruning accuracy loss to under 0.25%, even at higher sparsity. This demonstrates that moderate to high sparsity can be achieved with minimal degradation when paired with fine-tuning, though the effect depends on layer-wise sparsity configuration. Sensitivity analysis (Figure 2) confirms that early layers are more sensitive, exhibiting sharper accuracy declines as sparsity increases. Later layers tolerate higher pruning ratios with little degradation. The effect is stronger for structured pruning e.g., at $\alpha = 0.6$, unstructured pruning retains $\sim 88\%$ accuracy in early layers, while structured drops to $\sim 60\%$.

Weight distributions (Figure 3) illustrate these patterns. The dense model shows a near-Gaussian distribution centered at zero. After unstructured pruning, the distribution becomes bimodal, reflecting removal of small-magnitude weights. Structured pruning preserves the Gaussian form but with reduced density due to channel removal. Grad-CAM visualizations (Figure 4) qualitatively support these findings. The dense model produces well-localized activations around the upper-left region of the object. Unstructured pruning

preserves similar focus, though with weaker, more diffuse activations, which recover post-fine-tuning. Structured pruning initially shifts attention to unrelated regions but partially recovers after fine-tuning, albeit with less precision. These results confirm that while unstructured pruning preserves the original behavior, structured pruning provides hardware efficiency at the cost of fine-grained spatial fidelity.

3.2. Quantization

3.2.1. PTQ & QAT

Table 2 reports the accuracies of our models trained under different quantization schemes. A clear trend emerges: There is no significant drop when we quantize to FP16 or BF16. As expected, there is a greater loss in accuracy when we quantize to INT4 as compared to INT8. QAT is able to bring INT8 accuracy near to baseline in just one epoch, however there is a degradation of model accuracy with QAT and INT4 even after 10 epochs. The only anomaly in our experiments was the VGG-11 which was surprisingly performing better on INT4 as compared to INT8. The reasons for this are discussed in subsequent sections.

3.2.2. SCALING LAWS

Figure 5 illustrates how model accuracy scales with quantization bit width across different architectures and quantization strategies. We observe a clear trend: accuracy improves with increasing bit precision. Extremely low-precision models (INT4) exhibit significant degradation under PTQ, but QAT substantially mitigates this loss for all ResNet variants. The improvement from QAT is more pronounced in INT4 than in INT8, although INT4 still lags considerably behind INT8 in absolute performance. This behavior reflects the point of diminishing returns: below INT8, the accuracy loss grows disproportionately compared to the additional memory or latency benefits. As the bit width increases to INT8 and FP16/BF16, the accuracy converges towards the FP32 baseline, reflecting the diminishing returns of higher precision. Interestingly, VGG-11 deviates from this general trend, achieving higher accuracy with INT4 QAT than with INT8 PTQ. This anomaly suggests that the model’s layer-wise sensitivity and activation distribution differ substantially from the residual architectures, a point explored in later analysis.

3.2.3. MIXED PRECISION QUANTIZATION

We report the accuracy, latency, and model size of VGG-11 under different quantization schemes in table 3. Surprisingly, the accuracy approximately remains the same for all experiments. This is made more prominent by fig 6. Meanwhile, the models size decrease as expected. There is no clear trend in the latency which could be because of different optimizations of our GPU/CPU at different bit widths.

3.2.4. OUTLIERS IN QUANTIZATION

Table 4 summarizes the percentage of clipping applied to the weights and activations, in what granularities, together with the accuracy achieved. The results show that, in our setup, percentile clipping provides no significant improvement over standard PTQ, regardless of the granularity at which it is applied, as all methods converge around an accuracy of approximately 46%. The visualizations of weights and activations in Figure 7 indicate the absence of pronounced long-tail distributions, explaining why percentile-based clipping does not yield any performance gains.

3.3. Comparison of Distillation Approaches

We first compared the performance of the SI model, with LS, and with three different KD techniques: LM, DKD, and Hints, against the VGG-16 teacher baseline. The final validation accuracies on CIFAR-100 are summarized in Table 5. As shown, all KD methods outperformed the SI baseline. Hints-based distillation achieved the highest student accuracy (67.73%), nearly matching the teacher’s performance. DKD also showed significant improvement (65.57%), surpassing standard LM (64.06%) and CRD (64.16%). Label Smoothing (61.92%) performed slightly worse than the independent student baseline.

3.3.1. PROBABILITY DISTRIBUTION SIMILARITY

We quantified the similarity between the teacher’s (T) and students’ (S) output probability distributions using average KL divergence ($KL(T||S)$) with $T = 4.0$ on the test set. LM resulted in student distributions closest to the teacher’s (lowest KL divergence) with 0.1178. The SI model showed the largest divergence of $KL=0.3670$. Hints, despite achieving high accuracy, produced distributions less aligned with the teacher than LM did ($KL=0.2466$).

3.3.2. LOCALIZATION KNOWLEDGE TRANSFER

We assessed the transfer of spatial attention using GradCAM visualizations and cosine similarity between teacher and student heatmaps. Hints-based distillation yielded GradCAM heatmaps most similar to the teacher’s, indicated by the highest mean cosine similarity (0.7323). LM and CRD showed lower similarity scores compared to even the SI baseline (0.5911) in this metric with 0.5240 and 0.5325 mean cosine similarity respectively. Visualizations for sample images (see Figure 8) corroborate these findings, showing the Hints student’s attention often mirroring the teacher’s focus more closely than other methods.

3.3.3. COLOR INVARIANCE TRANSFER WITH CRD

We evaluated model robustness to color variations by comparing performance on the standard test set versus a color-

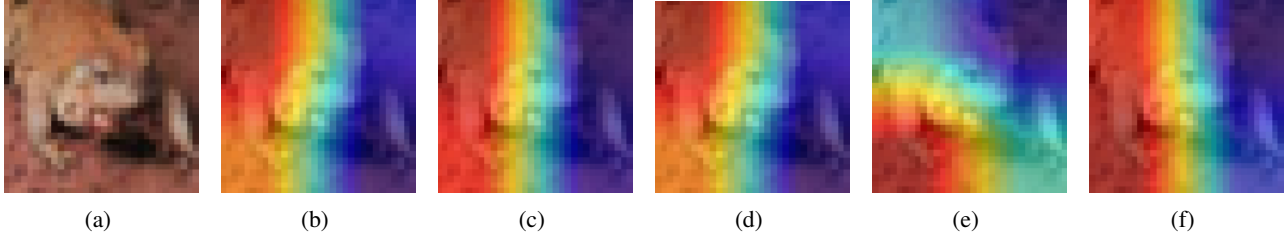


Figure 4. Grad-CAM visualizations. (a) original image, (b) dense model, (c) unstructured pruned (pre-finetune), (d) unstructured pruned (post-finetune), (e) structured pruned (pre-finetune), (f) structured pruned (post-finetune).

Table 2. Top-1 accuracy (%) on CIFAR-100 for reduced-precision and quantized models.

Model	FP32	FP16	BF16	INT8 PTQ	INT4 PTQ	INT8 QAT	INT4 QAT
VGG-11	70.80	70.83	70.73	46.44	61.12	70.2	62.64
ResNet-32	70.14	70.13	70.19	68.49	11.66	70.06	44.79
ResNet-44	71.66	71.62	71.55	67.66	11.74	71.55	44.81
ResNet-56	72.61	72.58	72.42	71.76	11.14	72.21	47.82

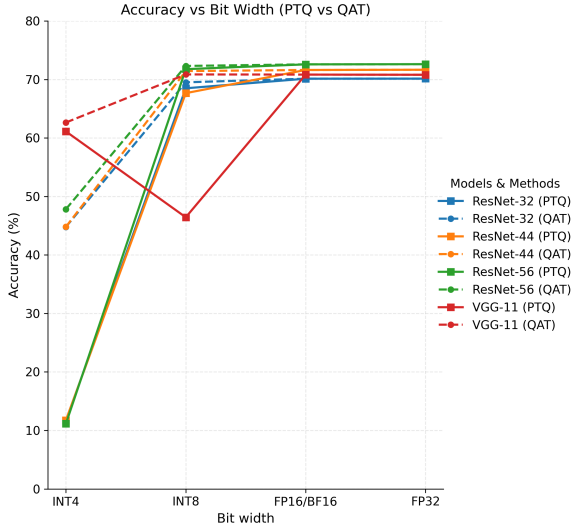


Figure 5. Accuracy vs. bit width for PTQ and QAT across architectures. Accuracy scales consistently with precision, with QAT narrowing the gap between quantized and FP32 models.

jittered version (Table 6). Fine-tuning the teacher with color jitter (T_{color}) significantly improved its robustness, reducing the accuracy drop from 8.40% to 3.68%. However, distilling from this color-invariant teacher using CRD (S_D) was less effective in transferring invariance to the student. While its jittered accuracy is better than all the previously distilled models (except Hints), The accuracy drop for S_D (8.96%) was comparable to that of the LM student (9.04%) and only slightly better than the SI student (10.82%) or the original CRD student (10.34%). Hints demonstrated the smallest accuracy drop among the standard KD students (8.11%), closest to the base teacher’s drop.

Table 3. VGG-11 on CIFAR-100 (CPU, FBGEMM). INT4 emul.; sizes in MiB.

Method	Acc. (%)	Th. Sz	Ser. Sz	Lat. (ms)
FP32	70.80	37.39	37.43	6.63
BF16	70.72	18.70	18.70	10.10
Mixed	70.92	9.40	9.48	3.51
Adaptive	70.55	7.99	9.43	2.81

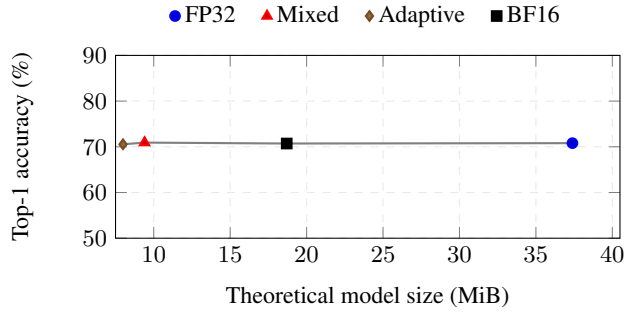


Figure 6. Accuracy vs. theoretical model size for VGG-11 on CIFAR-100 (CPU, FBGEMM).

3.3.4. EFFICACY OF A LARGER TEACHER

We compared the performance of VGG-11 students trained via LM using VGG-16 and 19 teachers. Despite VGG-19 having more parameters, the VGG-16 teacher yielded a slightly better student model (64.06% vs. 63.80%) using LM under the training regime. The VGG-19 teacher itself also achieved a slightly lower accuracy (67.45%) than the VGG-16 teacher (67.84%) after fine-tuning.

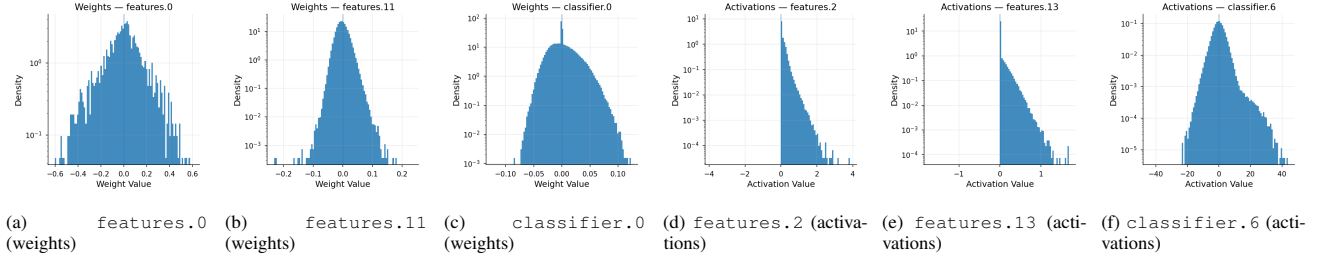


Figure 7. Histogram distributions for selected layers in VGG-11-BN (CIFAR-100). Left to right: weights (features.0, features.11, classifier.0) and activations (features.2, features.13, classifier.6). Y-axis is density (log scale).

Table 4. INT8 PTQ on VGG-11-BN (CIFAR-100). Percentile clipping is chosen via an MSE proxy between FP32 and INT8 logits on a 1k-image balanced calibration subset. $\uparrow/\downarrow$ denote higher/lower is better.

Method/Partition	Act. p^* (%)	Wgt. p^* (%)	Top-1 (%) $\uparrow$
FP32	—	—	70.80
INT8	—	—	46.44
INT8 Percentile	99.95	99.90	46.31
Clip-EARLY	99.95	99.90	46.24
Clip-MIDDLE	99.95	99.90	46.81
Clip-LATE	99.95	99.90	46.42

Model	Accuracy (%)
Teacher (VGG-16)	67.84
Student Independent (SI)	62.16
Label Smoothing (LS)	61.92
Logit Matching (LM)	64.06
Decoupled KD (DKD)	65.57
Hints	67.73
CRD	64.16

Table 5. Comparison of final validation accuracies for all distillation methods.

4. Discussion

4.1. Pruning

Our empirical results show that the VGG-11 architecture can be pruned extremely aggressively while achieving high degrees of sparsity and acceleration without large performance drops. Our results are supported by previous literature suggesting these trends of achieving high degrees of sparsity (90%) without compromising on accuracy (Frankle & Carbin, 2019) and can sometimes even improve results, as evident by the increase in accuracy of the fine-grained pruned model (Gale et al., 2019). The Grad-CAM visualizations further support post-pruning fine-tuning: after fine-tuning, the unstructured model nearly recovers the dense model’s attention, while the structured model’s maps are still slightly smoother and less localized. Furthermore, the curve observed in our sparsity vs accuracy graph suggests that the decrease is flat initially, followed by a sharp decline after a certain threshold. The sensitivity analysis plots fur-

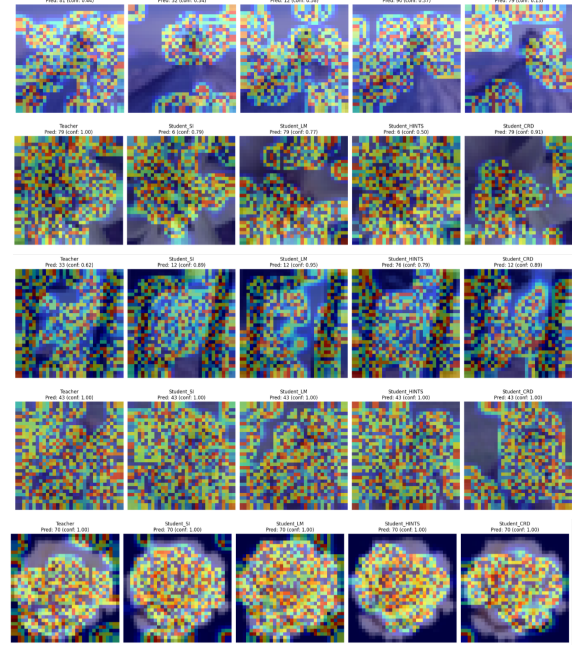


Figure 8. Grad-CAM visualizations for all sample images. Each shows model focus for a different true label.

ther explain this by suggesting that earlier layers are more sensitive to both types of pruning as compared to later and deeper layers. This is corroborated by research done into structured (channel) pruning papers, which highlight that earlier layers act as low-level feature extractors and should not be pruned aggressively, while deeper layers do not share this sensitivity (Li et al., 2017; Molchanov et al., 2017). Our structured results, where accuracy falls more steeply for the same nominal prune ratio, also agree with these reports since removing an entire channel is a stronger perturbation than zeroing a few weights inside it.

It is also important to analyze the differences between structured and unstructured pruning. Our results highlight that unstructured pruning has better task performance and incurs essentially no loss after fine-tuning but this is at the cost of no reduction in MACs or latency. However, structured pruning offers greater acceleration at the cost of lower task

performance. Thus, channel-based pruning is more useful for hardware acceleration and performance gains, which is highlighted in (He et al., 2017), which utilize structured pruning techniques to achieve speed-ups with minimal performance degradation.

To fully utilize the advantage of unstructured pruning, there need to be optimizations at the hardware level, which handle sparse matrices. This is done by sparse-aware kernels like cuSPARSE, MKL sparse. However, normal GPUs etc, which are not specialized for sparsity cannot achieve FLOP reduction, which is caused by irregular sparsity patterns and poor cache utilization as a result of unstructured pruning (Gale et al., 2019). Thus, our unstructured model stays at 606M MACs and nearly the same measured latency. Structured pruning, in contrast, preserves tensor shapes that dense libraries already optimize for (smaller channel dimensions, thinner layers), so the theoretical reduction is much more likely to be noticed in practice.

Model	Normal Acc.	Jittered Acc.	Drop (%)
Teacher (base)	0.6784	0.5944	8.40
Teacher (color-invariant)	0.6843	0.6475	3.68
Student_SI	0.6216	0.5134	10.82
Student_LM	0.6406	0.5502	9.04
Student_HINTS	0.6773	0.5962	8.11
Student_CRD	0.6416	0.5382	10.34
Student_SD (color-CRD)	0.6422	0.5526	8.96

Table 6. Performance comparison on standard vs. color-jittered test sets.

4.2. Quantization

4.2.1. PTQ & QAT

The recovery trends across bit-widths align with prior findings on QAT. Our INT8 model reached near-baseline accuracy within a single epoch, consistent with (McKinstry et al., 2019), where several 8-bit networks surpassed FP32 baselines after minimal fine-tuning on ImageNet. This rapid convergence arises from using the full training loader, providing sufficient data diversity for observers and batch normalization statistics to recalibrate effectively.

In contrast, INT4 quantization showed persistent degradation even after ten QAT epochs. Its limited representational capacity constrains both activations and weights, causing quantization noise to dominate gradient updates. As shown in (Chen et al., 2025), INT4 model saturate below BF16 (and FP32) accuracy, reflecting a precision-imposed upper bound rather than an optimization issue.

FP16 and BF16, by comparison, exhibit negligible degradation from FP32, differing only by minor mantissa rounding. Quantized INT models, however, discretize values into uniform bins, 256 for INT8 and just 16 for INT4, introducing noise inversely proportional to bit-width. While INT8 offers adequate resolution for most feature variations, INT4’s coarse granularity collapses subtle differences into shared

bins, irreversibly reducing expressive power. The degradation observed in our INT4 experiments thus stems from inherent structural limits of ultra-low-bit quantization, consistent with prior scaling law analyses. (Chen et al., 2025)

4.2.2. MIXED PRECISION QUANTIZATION

Our results showed that mixed-precision with higher precision in the early and late layers was the most effective. In VGG-11, keeping the first and last layers in BF16 and quantizing the middle to INT8 yields 70.92%, essentially identical to FP32 (70.80%) and BF16 (70.72%), while substantially reducing model size (Table 3). Both mixed schemes (“Mixed” and “Adaptive MP”) also outperform uniform INT PTQ on VGG-11 by a wide margin (INT8 PTQ: 46.44%, INT4 PTQ: 61.12%), showing that selective precision assignment is the dominant factor.

There are diminishing returns once the most sensitive layers are protected. The adaptive configuration (BF16/INT8/INT4) attains 70.55%, that is nearly the same as the simpler mixed setup (70.92%), despite finer layer-wise tuning and a smaller theoretical size. This indicates that additional precision adjustments beyond the first and last layers contribute little to accuracy.

The “Mixed” configuration and “Adaptive MP” (Hessian-guided) configuration ended up performing almost the same, even though the Adaptive approach was designed to be more informed. The reason is that the mixed setup already assigned higher precision (BF16) to exactly the same layers that the Hessian analysis later identified as most sensitive, the first and last layers. We ran a control ablation removing BF16 from the first layer which caused accuracy to collapse to $\approx 46\%$, confirming that early-layer precision is decisive.

4.2.3. OUTLIER BEHAVIOR IN QUANTIZATION

Percentile-based outlier suppression provided no measurable benefit across layers (Table 4). All clipping configurations, early, middle, and late, converged to similar accuracy ($\sim 46\%$), indicating that activation truncation had negligible effect. This aligns with the observed activation and weight distributions (Figure 7), which are smooth and lack pronounced long tails. Batch normalization stabilizes activation ranges, and limited ReLU saturation further prevents large-magnitude spikes, eliminating conditions where clipping would help.

Outlier clipping only improves quantization when extreme values distort the dynamic range, compressing most activations into narrow bins. In our setup, the absence of such skew means clipping merely rescales already well-behaved distributions without improving representational fidelity. Thus, percentile clipping offers little benefit when normalization enforces range consistency across channels.

However, the situation differs in larger architectures. Vision Transformers and large-scale language models often exhibit heavy-tailed activation statistics, particularly in attention and feedforward blocks where normalization is weaker and sparsity is higher. In such settings, a few dominant channels can inflate the quantization scale, severely limiting dynamic range. Robust quantization therefore requires adaptive range scaling, logarithmic quantizers, or channel-wise clipping that explicitly account for non-uniform activation distributions. As model depth and width grow, resilience to statistical extremes becomes essential for stable low-precision deployment.

4.3. Knowledge Distillation

Our experiments reveal a key trade-off: the method that best improves classification accuracy is not the one that best mimics the teacher’s output distribution. Analyzing accuracy (Table 5), all KD methods surpassed the Student Independent (SI) baseline (62.16%). Hint-based distillation (67.73%) was superior, nearly matching the teacher (67.84%). This stems from its loss signal: Hints enforces a dense, spatial regression between intermediate student and teacher layers. This acts as a strong structural regularizer, causing the student to replicate the teacher’s feature extraction process. In contrast, LM (64.06%) and CRD (64.16%) achieved smaller gains. LM applies its loss only at the final layer, teaching what to predict (logits) without guiding how to form internal features. CRD, while using internal features, targets the final 4096-dim representation. Its contrastive (NT-Xent) loss aligns global similarity, not spatial correspondence, providing a weaker signal than the pixel-wise regression of Hints. Thus, for the VGG-11-from-VGG-16 setup, the most transferable knowledge for accuracy is the teacher’s internal, spatial feature-extraction strategy. When evaluating output probability mimicry the trend reversed. LM was the best mimic (KL: 0.1178), as its objective directly minimizes the KL divergence between logits. Conversely, Hints (KL: 0.2466) and SI (KL: 0.3670) were poor mimics. Hints optimizes at earlier features, not the final logits; its head is trained with L_{CE} on hard labels. The student learns the teacher’s internal process but forms its own decision boundary, which is highly accurate but not logit-aligned. SI, trained only on hard labels, becomes overconfident and lacks dark knowledge, resulting in the highest divergence. This trade-off is central: LM produces the best mimic, while Hints yields the best performer. The knowledge driving student accuracy (internal representations) differs from that driving student fidelity (output logits).

4.3.1. ANALYSIS OF LOCALIZATION KNOWLEDGE TRANSFER

We analyzed spatial attention transfer by comparing Grad-CAM heatmaps. Quantitatively (Table ??), only Hint-based

distillation (0.7323) significantly improved localization similarity over the SI baseline (0.5911). This outcome directly reflects the method’s design: Hint-based KD applies a spatially-aware MSE loss between intermediate convolutional layers. Because this loss acts on feature maps *before* spatial information is collapsed, the student learns to replicate the teacher’s spatial activations. Since GradCAM is derived from these same maps, the high cosine similarity (0.7323) follows naturally. In contrast, LM (0.5240) and CRD (0.5325) performed worse than SI, consistent with their objectives, LM constrains only the final 100-dimensional logits, and CRD operates on a flattened 4096-dimensional vector, both devoid of 2D spatial structure. Without a spatial training signal, LM and CRD backbones behave similarly to the SI model trained solely with L_{CE} . Visualizations (Figure 8) corroborate this: T and Hints show focused, structurally similar heatmaps, while SI, LM, and CRD exhibit diffuse noise. Notably, T and Hints produce nearly identical diffuse regions even when predicting different incorrect classes (33 and 76), indicating that Hints effectively transfers the teacher’s spatial reasoning—independent of final classification outcomes.

4.3.2. ANALYSIS OF COLOR INVARIANCE TRANSFER

This experiment tested whether a student could implicitly inherit a teacher’s invariance to color transformations. As a premise check, fine-tuning the base teacher (T_{base}) with `ColorJitter` produced a robust variant (T_{color}), reducing the jittered-data accuracy drop from 8.40% to 3.68% (Table 6). Distillation from this invariant teacher using CRD yielded partial but inefficient transfer. The distilled student (S_D) reached 55.26% jittered accuracy, above both S_{SI} (51.34%) and baseline S_{CRD} (53.82%), indicating some transfer of invariance. However, its drop (8.96%) remained comparable to S_{LM} (9.04%) and far worse than the teacher’s 3.68%. This inefficiency stems from CRD’s design: it aligns global representations for a single static input, whereas invariance is a functional property that maps multiple augmented views to similar embeddings. Because S_D was trained only on non-jittered data, it replicated T_{color} ’s static features rather than its robust mapping, resulting in weak, implicit transfer. Notably, the Hints student (59.62%) remained the most robust under jitter despite training with the non-invariant T_{base} . This aligns with earlier findings: by regressing dense, spatially-rich feature maps, Hints captures more abstract, color-agnostic representations, transferring greater robustness than CRD, even without explicit invariance supervision.

4.3.3. ANALYSIS OF TEACHER CAPACITY

This experiment tested the assumption that a larger, more powerful teacher yields a better student. Our earlier results for the efficacy of a larger teacher contract the hypothesis

for this setup. The VGG-11 student distilled from the larger The VGG-19 teacher (63.80%) performed slightly worse than the student distilled from the The VGG-16 teacher (64.06%). This aligns with the teachers' own performance: The VGG-16 achieved 67.84% versus VGG-19's 67.45%. Thus, the efficacy of LM depends not on the teacher's capacity but on the quality and suitability of its learned distribution. LM compels the student to replicate the teacher's output probabilities, faithfully inheriting both its strengths and flaws. If a larger teacher is slightly sub-optimal, perhaps due to overfitting or poor fine-tuning on CIFAR-100, the student will inherit this sub-optimal knowledge. Here, the cleaner, better-optimized knowledge from VGG-16 offered a stronger supervisory signal than the marginally less-performant VGG-19. This highlights that teacher selection is critical and bigger is not necessarily better.

4.4. Comparison

4.4.1. CONVENIENCE AND PRACTICALITY

In terms of implementation ease, the methods vary significantly. PTQ is by far the most convenient, acting as a simple post-processing step that requires no retraining. This contrasts sharply with pruning, which holds moderate complexity due to its iterative cycle of training, masking, and fine-tuning to recover accuracy. The most complex methods are KD and QAT, as both demand full, specialized retraining pipelines. KD is particularly involved, requiring the management of two models and careful tuning of loss functions, where the choice (Hint vs. LM etc) impacts performance.

4.4.2. POTENTIAL DRAWBACKS AND ADVANTAGES

Each technique presents a unique risk-reward profile. Pruning's main advantage comes from structured pruning, which physically removes channels to create a smaller, dense model that accelerates inference on any hardware. But this is at the cost of lower task performance, which creates a tradeoff as shown in Table 1. Unstructured pruning while achieving high degrees of sparsity with no accuracy loss, offers no practical speed-up on standard hardware without specialized sparse-operation libraries, which creates a major drawback. Quantization's chief advantage is it being extremely hardware-optimized delivering the most significant inference acceleration by leveraging optimized, low-precision instructions on modern hardware like GPUs and NPUs. Its greatest weakness, however, is outliers, which can massively hurt performance. Finally, KD's advantage is unique. It can offer improvement over the teacher with reduced memory footprint and latency, but this comes at the cost of a high training budget and a strong dependence on teacher quality. This dependence was clear as we show a larger but sub-optimal VGG-19 teacher produced a worse student than the VGG-16.

4.4.3. EFFECTIVENESS ON A FIXED COMPUTE BUDGET

The optimal technique depends entirely on the nature of the budget. If the budget is for training, PTQ is the cheapest, while KD is the most expensive due to the overhead of the teacher model. However, if the goal is achieving the highest accuracy for a fixed-size model, KD is the most effective. Pruning and Quantization are subtractive processes that trade accuracy for efficiency. KD is an additive process that transfers dark knowledge to create a stronger and robust baseline model, as exemplified by our Hints-based student.

4.4.4. RECOMMENDATION

An effective pipeline should not rely on a single technique but rather should ensemble them. We suggest that a good pipeline would first distill knowledge to create a highly accurate student model based on the memory budget. The resulting small model would be quantized to INT8/4 or MP for maximum hardware acceleration. This combination is effective since distillation would indirectly also reduce latency and memory overhead, compared to the teacher, and quantization will maximize performance on hardware. While quantization offers the greatest raw inference speed-up, KD is the most effective technique. As our experiments confirmed, a student model that learns a teacher's internal representations or output distributions becomes a far superior starting point for subsequent pruning and quantization than a model trained from scratch on hard labels alone. In essence KD, does the work of pruning already by reducing memory footprint and latency and can offer gains through alternative routes than quantization.

5. Conclusion

We demonstrate that model compression effectively optimizes deep learning models by reducing memory and computation overhead. On CIFAR-10, CNNs show substantial redundancy: unstructured pruning to $\sim 70\%$ sparsity with brief fine-tuning recovers baseline accuracy, while structured pruning achieves real MAC and latency reductions at minor accuracy and localization cost. For quantization, sensitivity-aware strategies enable large compression with minimal degradation. On CIFAR-100, distilling a VGG-16 teacher to VGG-11 revealed that internal feature matching (Hints) yields the best accuracy (67.73%), whereas output-logit matching achieves higher fidelity (KL: 0.1178) but lower performance. Overall, combining pruning, quantization, and distillation provides the best trade-off between acceleration, efficiency, and accuracy retention.

References

Chen, M., Zhang, C., Liu, J., Zeng, Y., Xue, Z., Liu, Z., Li, Y., Ma, J., Huang, J., Zhou, X., and Luo, P. Scaling

- law for quantization-aware training, 2025. URL <https://arxiv.org/abs/2505.14302>.
- Dettmers, T. and Zettlemoyer, L. The case for 4-bit precision: k-bit inference scaling laws, 2023. URL <https://arxiv.org/abs/2212.09720>.
- Dong, Z., Yao, Z., Cai, Y., Arfeen, D., Gholami, A., Mahoney, M. W., and Keutzer, K. Hawqv2: Hessian aware trace-weighted quantization of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)* 33, pp. 4157–4167, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/d77c703536718b95308130ff2e5cf9ee-Paper.pdf>.
- Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019. URL <https://arxiv.org/abs/1803.03635>.
- Gale, T., Elsen, E., and Hooker, S. The state of sparsity in deep neural networks, 2019. URL <https://arxiv.org/abs/1902.09574>.
- Han, S., Pool, J., Tran, J., and Dally, W. J. Learning both weights and connections for efficient neural networks, 2015. URL <https://arxiv.org/abs/1506.02626>.
- He, Y., Zhang, X., and Sun, J. Channel pruning for accelerating very deep neural networks, 2017. URL <https://arxiv.org/abs/1707.06168>.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>. Technical Report.
- Krizhevsky, A. Learning multiple layers of features from tiny images. *University of Toronto*, 05 2012.
- Li, H., Kadav, A., Durdanovic, I., Samet, H., and Graf, H. P. Pruning filters for efficient convnets, 2017. URL <https://arxiv.org/abs/1608.08710>.
- McKinstry, J. L., Esser, S. K., Appuswamy, R., Bablani, D., Arthur, J. V., Yildiz, I. B., and Modha, D. S. Discovering low-precision networks close to full-precision networks for efficient embedded inference, 2019. URL <https://arxiv.org/abs/1809.04191>.
- Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. Pruning convolutional neural networks for resource efficient inference, 2017. URL <https://arxiv.org/abs/1611.06440>.
- Or, A., Jain, A., Vega-Myhre, D., Cai, J., Hernandez, C. D., Zhang, Z., Guessous, D., Kuznetsov, V., Puhersch, C., Saroufim, M., Rao, S., Tran, T., and Samardžić, A. Torchao: Pytorch-native training-to-serving model optimization, 2025. URL <https://github.com/pytorch/ao>. arXiv preprint arXiv:2507.16099.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 128 (2):336–359, October 2019. ISSN 1573-1405. doi: 10.1007/s11263-019-01228-7. URL <http://dx.doi.org/10.1007/s11263-019-01228-7>.
- Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition, 2015. URL <https://arxiv.org/abs/1409.1556>.
- Yao, C. Pytorch cifar models: Pretrained resnet, vgg, etc. on cifar-10/100. <https://github.com/chenyaofo/pytorch-cifar-models>, 2021. Accessed: YYYY-MM-DD.